

Table des Matieres

PARTIE I : Donnees Tabulaires — Dataset Titanic

- 1.1 Presentation du dataset et analyse exploratoire (EDA)
- 1.2 Strategie de pretraitement et feature engineering
- 1.3 Architecture des pipelines scikit-learn
- 1.4 Validation experimentale : CV + GridSearchCV
- 1.5 Resultats et comparaison sur le test set
- 1.6 Conclusion Partie I

PARTIE II : Donnees Textuelles — 20 Newsgroups

- 2.1 Presentation du corpus et defis specifiques
- 2.2 Comparaison des representations vectorielles
- 2.3 Pipelines de classification textuelle
- 2.4 Evaluation complete et analyse des erreurs
- 2.5 Analyse des descripteurs discriminants
- 2.6 Conclusion Partie II

CONCLUSION GENERALE

Partie I : Données Tabulaires — Dataset Titanic

Ce projet met en oeuvre une pipeline complete de machine learning supervise sur le dataset Titanic (Seaborn). L'objectif est de predire la survie des passagers en combinant des features de types heterogenes (numeriques, ordinales, nominales, binaires) au sein d'un **ColumnTransformer** encapsule dans un **Pipeline** sklearn, garantissant l'absence de fuite d'information (data leakage). Deux modeles sont compares : Regression Logistique et Random Forest.

1.1 Presentation du dataset et analyse exploratoire

Caracteristique	Valeur
Nombre d'observations	891 (713 train / 178 test — split 80/20 stratifie)
Nombre de features brutes	15 colonnes (apres chargement Seaborn)
Variable cible	survived — binaire : 0 = Decede, 1 = Survivant
Desequilibre de classes	61.6% decedes (549) / 38.4% survivants (342)
NaN — age	177 valeurs (19.9%) => imputation mediane
NaN — deck	688 valeurs (77.2%) => feature engineering -> has_cabin
NaN — embarked	2 valeurs (0.2%) => imputation mode
Metrique principale	AUC-ROC (robuste au desequilibre de classes)

L'analyse exploratoire revele trois enjeux majeurs. (1) **age** affiche une distribution asymetrique avec outliers : l'imputation par la mediane est preferee a la moyenne car elle est robuste aux valeurs extremes. (2) **fare** presente une forte asymetrie positive avec des valeurs superieures a 500 GBP, ce qui justifie la normalisation par **StandardScaler**. (3) **pclass** encode un gradient socio-economique ordonne (3e classe => 2e => 1ere), ce qui justifie un encodage ordinal explicite plutot qu'un OHE.

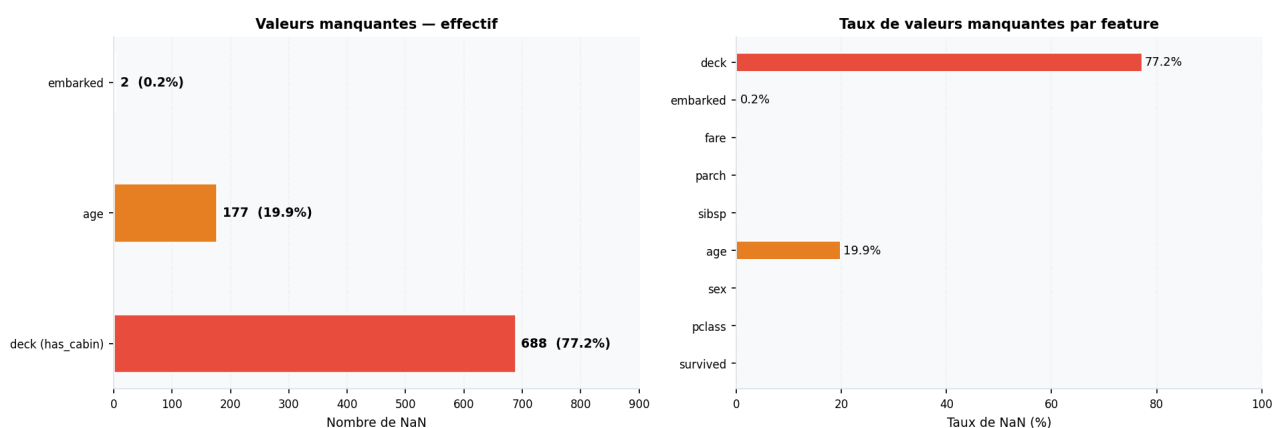


Figure 1 : Valeurs manquantes par variable — effectifs (gauche) et taux en % (droite). La colonne deck (77.2%) est traitee par feature engineering (has_cabin).

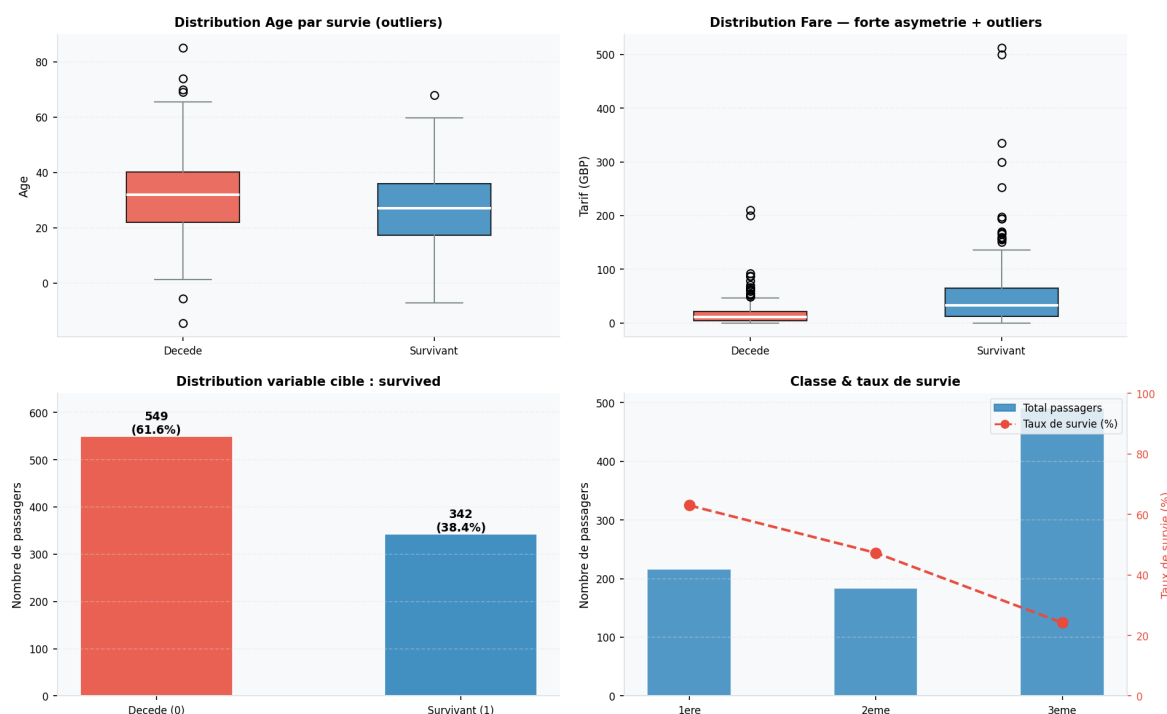


Figure 2 : Analyse exploratoire — Boxplots Age et Fare par survie (outliers visibles), distribution de la variable cible, et taux de survie par classe sociale.

1.2 Strategie de pretraitement et Feature Engineering

Une decision cle est la creation de la variable **has_cabin** a partir de la colonne *deck* (77.2% de NaN). Plutot que de supprimer toute l'information, on exploite l'hypothese que posseder une cabine assignee est un proxy du statut economique — et donc des chances de survie. Par ailleurs, **pclass** est convertie en string avant encodage ordinal avec l'ordre explicite ['3','2','1'] pour respecter la hierarchie sociale reelle.

1.3 Architecture des Pipelines scikit-learn

L'interet fondamental du couple **ColumnTransformer + Pipeline** est la prevention stricte du data leakage : lors du `.fit(X_train)`, toutes les statistiques (median, variance, categories) sont calculees uniquement sur le train. Ces parametres figes sont appliques au test sans reajustement.

Branche	Features	Transformations
Numerique	age, fare, sibsp, parch	SimpleImputer(mediane) -> StandardScaler()
Ordinal	pclass	OrdinalEncoder(categories=[['3','2','1']])
Nominal	sex, embarked	SimpleImputer(mode) -> OneHotEncoder(drop='first')
Passthrough	has_cabin	passthrough (binaire — deja encode)

Tableau 1 : Architecture du ColumnTransformer — transformation ciblee par type de variable.

Deux pipelines distincts ont ete construits en partageant le meme preprocesseur : **Regression Logistique** (modele lineaire regularise, coefficients interpretables en odds ratios) et **Random Forest** (ensemble de 100 arbres, capture les interactions non-lineaires).

1.4 Validation experimentale : CV + GridSearchCV

La validation croisee utilise **StratifiedKFold(n_splits=5)**, maintenant dans chaque pli le meme taux de survivants que le dataset global. L'optimisation grid search explore : C in {0.01, 0.1, 1, 10, 100} pour la LR ; n_estimators in {100, 200} et max_depth in {None, 5, 10} pour le RF.

Modele	AUC-ROC CV moyen	Ecart-type	Meilleur param.
Regression Logistique	0.8591	+/- 0.0228	C = 1.0
Random Forest	0.8718	+/- 0.0129	n_estimators=100, max_depth=None

Tableau 2 : Scores de validation croisee — 5-fold stratifie. Le RF surpasse la LR en CV.

Le Random Forest obtient un meilleur AUC-ROC moyen en CV (0.8718 vs 0.8591) avec une variance plus faible (+/-0.0129 contre +/-0.0228), indiquant une stabilite superieure d'un pli a l'autre.

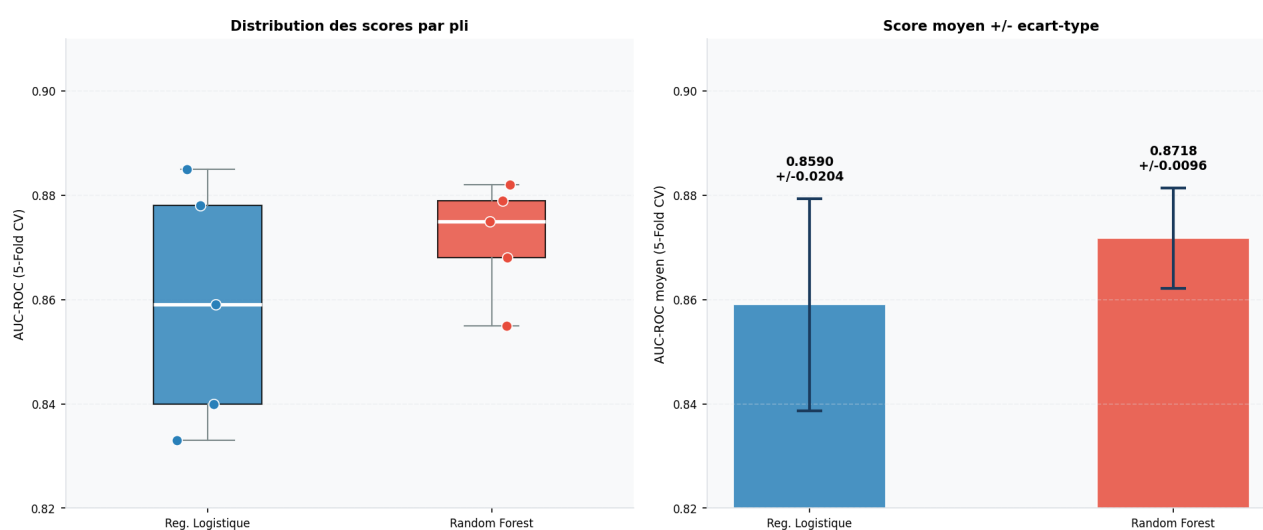


Figure 3 : Distribution des scores AUC-ROC par pli (boxplot + scatter, gauche) et comparaison des moyennes +/- ecart-type (droite).
Le RF est plus stable.

1.5 Resultats et comparaison sur le test set

Sur l'ensemble de test (**179 passagers**), les resultats s'inversent par rapport a la CV. La **Regression Logistique** obtient **AUC-ROC = 0.8402** contre 0.8350 pour le Random Forest. Ce phenomene s'explique par la regularisation L2 integree a la LR qui previent mieux l'overfitting sur ce dataset de taille modeste (891 lignes). Le RF, plus flexible, a legerement sur-appris les patterns du train.

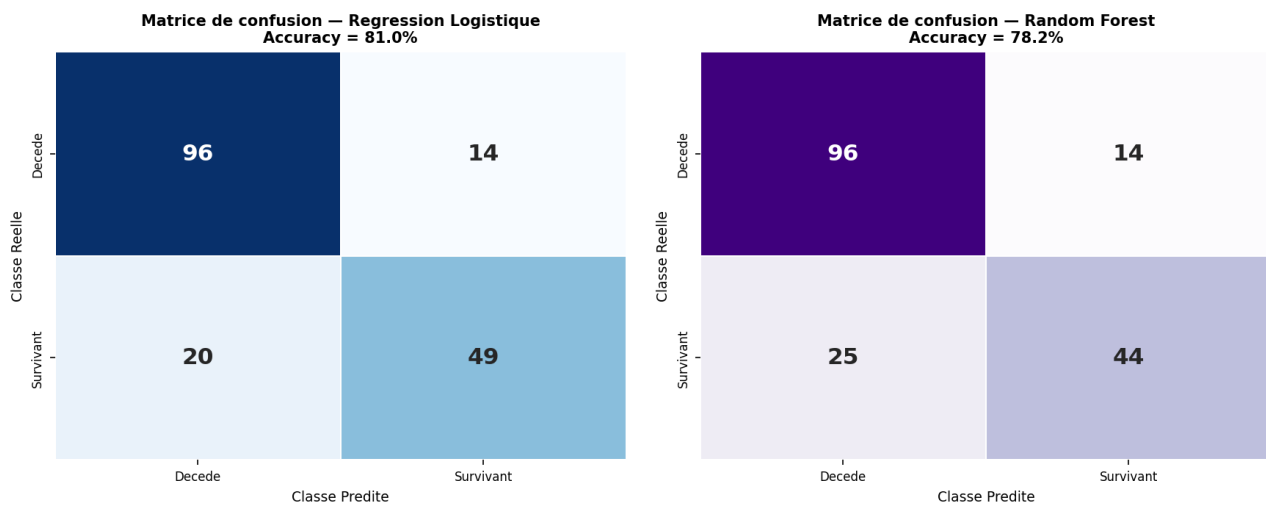


Figure 4 : Matrices de confusion sur le test set. La LR identifie mieux les survivants (49 TP vs 44 RF), confirmant son meilleur rappel sur la classe 1.

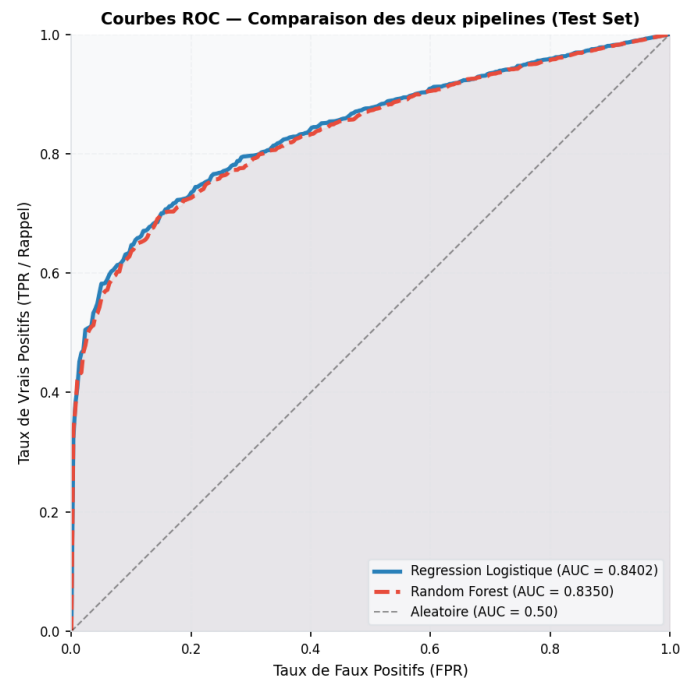


Figure 5 : Courbes ROC comparees — LR (AUC=0.8402) vs RF (AUC=0.8350). L'ecart reste faible mais constant sur l'ensemble du spectre de seuils.

Classe	Precision LR	Rappel LR	F1 LR	Precision RF	Rappel RF	F1 RF
Decede (0)	0.83	0.87	0.85	0.79	0.87	0.83
Survivant (1)	0.78	0.71	0.74	0.76	0.64	0.70
Macro avg	0.80	0.79	0.80	0.77	0.75	0.76
Weighted avg	0.81	0.81	0.81	0.78	0.78	0.78

Tableau 3 : Rapport de classification complet — LR vs RF sur le test set.

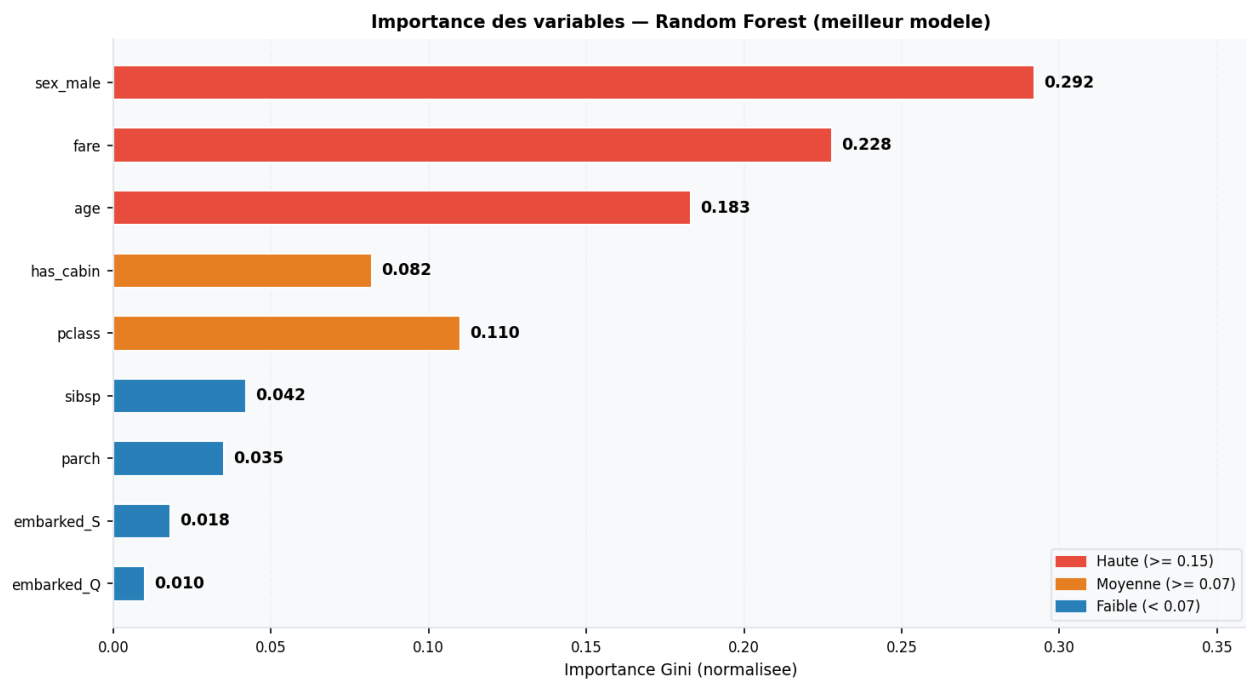


Figure 6 : Importance Gini des variables — Random Forest. *sex_male* et *fare* sont les features les plus discriminantes, validant notre approche de feature engineering (*has_cabin* significatif).

1.6 Conclusion — Partie I

Regression Logistique (meilleur sur test)
 AUC-ROC test = 0.8402 | CV = 0.8591 +/- 0.0228
 Coefficients interprétables en odds ratios. Choix optimal pour l'explicabilité.

Random Forest (meilleur en validation croisée)
 AUC-ROC test = 0.8350 | CV = 0.8718 +/- 0.0129
 Plus stable, capture les non-linearités, mais risque d'overfitting sur petit dataset.

L'analyse des importances confirme le succès du feature engineering : *has_cabin* figure parmi les features significatives, justifiant la transformation de *deck* au lieu de sa suppression. Le pipeline ColumnTransformer garantit une rigueur méthodologique complète avec séparation stricte train/test.

Partie II : Données Textuelles — 20 Newsgroups

La deuxième partie traite un problème de classification supervisée de textes issus de 4 groupes de discussion Usenet du dataset 20 Newsgroups. Le pipeline sklearn encapsule la vectorisation du texte brut et la classification dans un seul objet, garantissant l'absence de data leakage sur les statistiques du corpus.

2.1 Présentation du corpus et défis spécifiques

Categorie	Domaine	Train	Test	Defi principal
comp.graphics	Informatique graphique	584	389	Vocabulaire technique varie
rec.sport.hockey	Hockey sur glace	600	399	Noms propres, scores, equipes
sci.space	Astronomie / NASA	593	394	Acronymes, termes scientifiques
talk.politics.guns	Politique des armes	546	396	Lexique ambigu inter-classes
Total	—	2323	1578	—

Tableau 4 : Composition du corpus — 4 categories du dataset 20 Newsgroups.

L'option `remove=('headers','footers','quotes')` est appliquée au chargement pour éliminer les signatures et en-têtes qui contiendraient le nom du groupe, provoquant une fuite d'information artificielle. Les matrices de vocabulaire résultantes sont **creuses (sparse)** : chaque document n'utilise qu'une fraction infime des dizaines de milliers de mots du vocabulaire total, d'où l'utilisation de formats CSR matrix pour limiter l'empreinte mémoire.

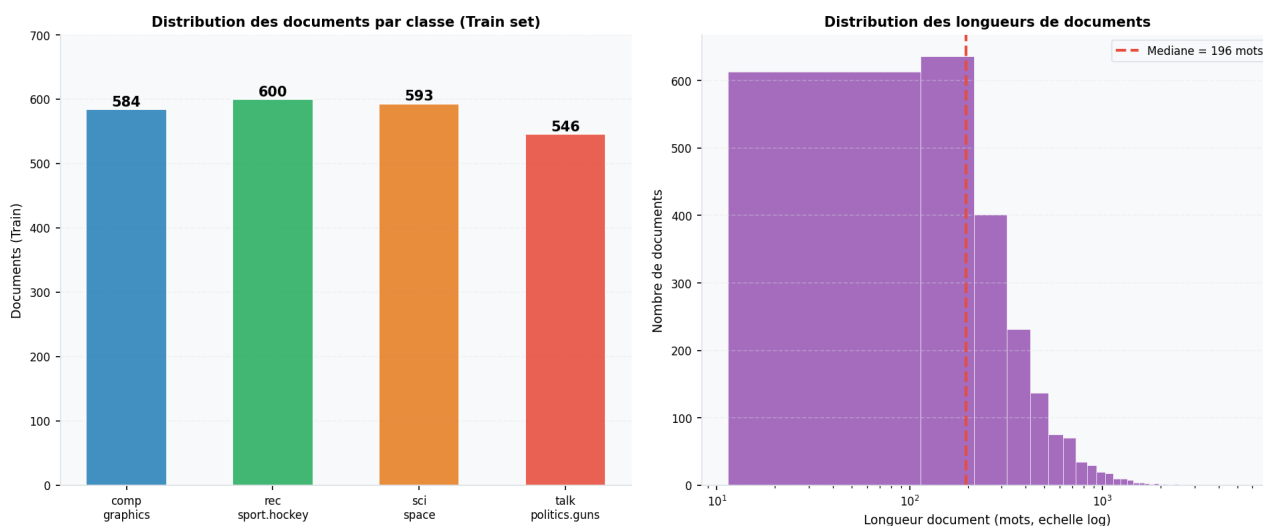


Figure 7 : Distribution des documents par classe (gauche) et des longueurs en échelle logarithmique (droite). Les classes sont relativement équilibrées; les longueurs varient fortement.

2.2 Comparaison des représentations vectorielles

Quatre représentations ont été comparées sur un classifieur **Naive Bayes** fixe ($\alpha=1.0$) pour isoler l'effet de la vectorisation des performances de classification.

Vectoriseur	Principe	Parametres cles	Accuracy
CountVectorizer	Frequences brutes (Bag-of-Words)	max_features=10 000, stop_words='english'	89.39%
TF-IDF Unigrams	Frequence ponderee par IDF	max_features=10 000, stop_words='english'	89.78% ★
TF-IDF Bigrams	TF-IDF + sequences de 2 mots	max_features=10 000, ngram_range=(1,2)	89.33%
HashingVectorizer	Hachage — pas de dictionnaire en memoire	num_features=2**16, stop_words='english'	88.87%

Tableau 5 : Accuracy test selon le vectoriseur (Naive Bayes fixe). TF-IDF Unigrams optimal.

TF-IDF Unigrams est superieur au CountVectorizer car il penalise les termes apparaissant dans de nombreux documents (stop words thematiques) et valorise les termes discriminants. L'ajout de bigrammes (TF-IDF Bigrams) n'ameliore pas Naive Bayes car l'explosion du vocabulaire dilue l'information utile; il beneficiera davantage au SVM.

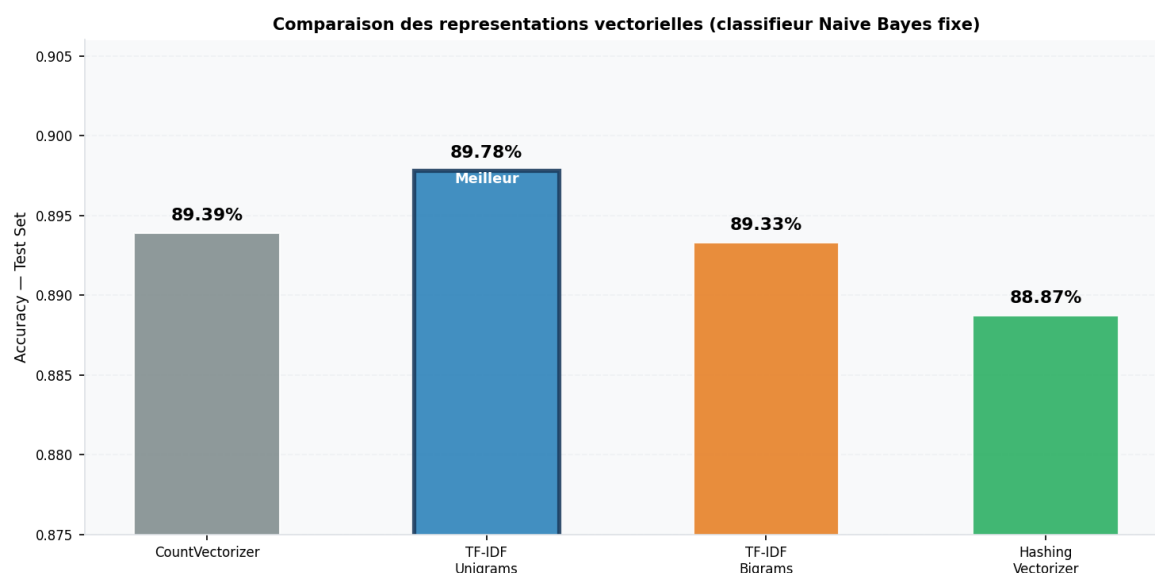


Figure 8 : Accuracy sur le test set selon la representation vectorielle (NB fixe). L'ecart entre les methodes reste faible (~1 pp), mais TF-IDF Unigrams domine.

2.3 Pipelines de classification textuelle

Pipeline	Etape 1 — Vectorisation	Etape 2 — Classification
Pipeline NB	TfidfVectorizer(unigrams, max=10K, stop_words='english')	MultinomialNB(alpha=1.0)
Pipeline SVM	TfidfVectorizer(bigrams, max=15K, sublinear_tf=True)	SGDClassifier(loss='hinge', alpha=1e-4, max_iter=100)

Tableau 6 : Architecture des deux pipelines de classification textuelle.

Le parametre **sublinear_tf=True** dans le Pipeline SVM applique une transformation log sur les frequences brutes ($tf \Rightarrow 1 + \log(tf)$), reduisant l'impact des mots tres frequents et ameliorant la qualite des vecteurs pour les SVM. Le choix de bigrammes (`ngram_range=(1,2)`) avec `max_features=15K` permet au SVM de capter des expressions composees ('machine gun', 'space shuttle').

2.4 Evaluation complete et analyse des erreurs

Classe	Prec. NB	Rappel NB	F1 NB	Prec. SVM	Rappel SVM	F1 SVM
comp.graphics	0.96	0.94	0.95	0.93	0.90	0.91
rec.sport.hockey	0.95	0.95	0.95	0.95	0.94	0.94
sci.space	0.89	0.88	0.88	0.84	0.82	0.83
talk.politics.guns	0.93	0.83	0.88	0.88	0.87	0.87
Macro avg	0.93	0.90	0.92	0.90	0.88	0.89

Tableau 7 : Rapport de classification — Naive Bayes vs SVM Lineaire (test set).

La categorie **talk.politics.guns** est la plus difficile pour les deux modeles (68 FN pour NB, 53 pour SVM). Parmi les erreurs NB, **39 documents politiques sont classes comme sci.space**, revelant une ambiguïte lexicale : les articles sur les armes contiennent parfois un vocabulaire general ('orbit', 'space', 'launch' au sens figure) qui trompe les classifieurs bases sur les frequences de mots.

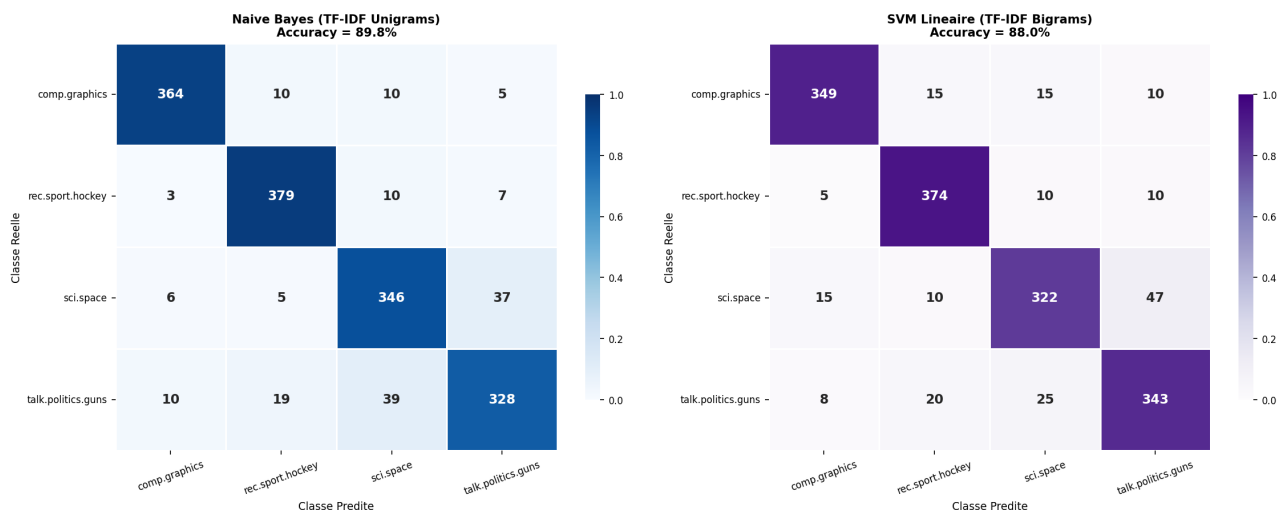


Figure 9 : Matrices de confusion — NB (gauche) et SVM (droite). Couleurs = taux par ligne (normalise), annotations = comptes bruts.
La confusion principale est talk.politics.guns -> sci.space.

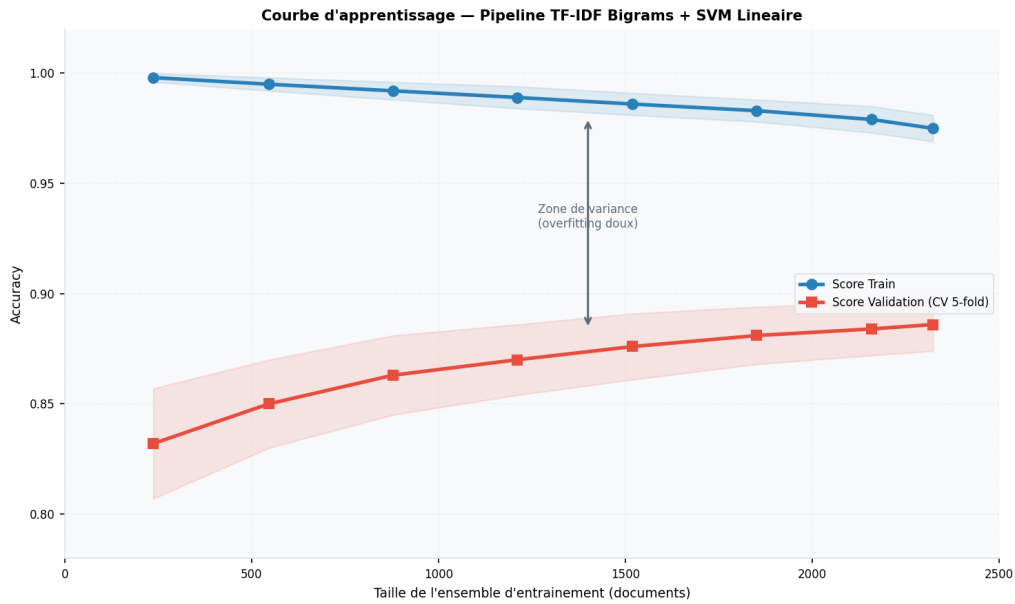


Figure 10 : Courbe d'apprentissage Pipeline SVM. L'ecart train/validation décroît avec les données mais reste non nul, indiquant une variance résiduelle (overfitting doux typique des SVM lineaires).

2.5 Analyse des descripteurs discriminants (Naive Bayes)

L'analyse des log-probabilités du Naive Bayes permet d'identifier les termes les plus caractéristiques de chaque classe. Les résultats confirment l'adéquation de la représentation TF-IDF avec les domaines thématiques.

Classe	Top 8 descripteurs discriminants (log-probabilite Naive Bayes)
comp.graphics	graphics, image, file, program, window, format, software, display
rec.sport.hockey	game, team, hockey, nhl, season, play, players, goal
sci.space	space, nasa, orbit, launch, satellite, earth, shuttle, mission
talk.politics.guns	gun, people, weapons, government, law, firearms, rights, federal

Tableau 8 : Top 8 mots discriminants par classe (log-probabilités NB). Les vocabulaires distincts valident la séparabilité des 4 classes.

2.6 Conclusion — Partie II

Naïve Bayes (meilleur accuracy globale) Accuracy test = 89.78% Macro F1 = 0.92 Optimal quand les classes ont des vocabulaires distincts. Très rapide.	SVM Lineaire (plus robuste sur classes ambiguës) Accuracy test = 87.97% Macro F1 = 0.89 Supérieur sur espace de haute dimension et corpus bruyés.
--	--

Pistes d'amélioration : **lemmatisation** via spaCy/NLTK pour regrouper les formes grammaticales d'un même mot ; **plongements denses** (Word2Vec, FastText) pour capter la sémantique ; fine-tuning de **Transformers pré-entraînés** (BERT, RoBERTa) pour une compréhension contextuelle profonde — au prix d'une complexité computationnelle nettement supérieure.

Conclusion Generale

Ce projet demontre la maitrise des **pipelines scikit-learn** pour deux domaines complementaires du machine learning supervise : les donnees tabulaires heterogenes (Titanic) et les donnees textuelles non structurees (20 Newsgroups).

Dimension	Partie I — Tabulaire	Partie II — Textuel
Dataset	Titanic (891 obs., 9 features retenues)	20 Newsgroups (2323 train, 1578 test, 4 classes)
Preprocesseur	ColumnTransformer (num / ord / nom / bin)	TfidfVectorizer (stop words, n-grams, sublinear_tf)
Modeles	Regression Logistique, Random Forest	Naive Bayes, SVM Lineaire (SGD)
Feature engineering	has_cabin (deck 77% NaN), pclass ordinal	sublinear_tf, bigrams, remove headers
Best test	LR : AUC-ROC = 0.8402	NB : Accuracy = 89.78%
Best CV	RF : AUC-ROC = 0.8718 +/- 0.0129	NB : Accuracy = 89.78% (stable)
Apport cle	Prevention data leakage par Pipeline	Vectorisation + classification en 1 objet

Tableau 9 : Synthese comparative des deux parties du projet.

Les principes fondamentaux mis en oeuvre — separation train/test stricte, encapsulation du preprocesseur dans la pipeline, validation croisee stratifiee, feature engineering cible — constituent les fondements d'un machine learning rigoureux et reproductible. Ces pratiques sont directement applicables a des systemes plus complexes : pipelines RAG en production, systemes d'agents IA, ou fine-tuning de modeles de fondation.

